

C01-AT3 (EXPLORATORY DATA ANALYSIS)

DSA0401-Fundamentals of Data Science

Name: C Nithisha

Reg.No:192424375

1. Education Analytics Dataset

Question 1

A college wants to analyze student performance based on attendance, study hours, internal marks, and final result. Perform EDA to identify the factors affecting student results.

Student_ID	Attendance	Study_Hours	Internal_Marks	Result
S01	92	5	45	Pass
S02	55	2	24	Fail
S03	78	4	38	Pass
S04	60	3	28	Fail
S05	88	5	42	Pass
S06	47	1	20	Fail
S07	95	6	48	Pass
S08	72	3	35	Pass
S09	50	2	22	Fail
S10	83	4	40	Pass
S11	66	3	30	Fail
S12	90	5	44	Pass
S13	58	2	25	Fail
S14	76	4	36	Pass
S15	62	3	29	Fail
S16	85	5	41	Pass
S17	45	1	18	Fail
S18	80	4	39	Pass
S19	68	3	32	Pass
S20	53	2	23	Fail

CODE:

```
from google.colab import files

# Upload CSV file
uploaded = files.upload()

# Get uploaded file name
filename = list(uploaded.keys())[0]

print("Uploaded file:", filename)

<IPython.core.display.HTML object>

Saving student_data.csv to student_data.csv
Uploaded file: student_data.csv

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Read CSV file
df = pd.read_csv(filename)

# Display first 5 rows
print("First 5 Records:")
display(df.head())

# Dataset information
print("\nDataset Information:")
df.info()

# Dataset size
print("\nNumber of Rows and Columns:")
print(df.shape)

# Statistical summary
print("\nStatistical Summary:")
display(df.describe())

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Check duplicate values
print("\nDuplicate Records:")
print(df.duplicated().sum())

# Correlation heatmap
plt.figure(figsize=(10,6))
sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
```

```

cmap="coolwarm"
)
plt.title("Correlation Between Student Performance Factors")
plt.show()

# Score distribution
numeric_columns = df.select_dtypes(include='number').columns

plt.figure(figsize=(8,5))
sns.histplot(
    df[numeric_columns[-1]],
    kde=True,
    color="blue"
)
plt.title("Student Performance Score Distribution")
plt.xlabel("Score")
plt.show()

# Correlation with final result/score
print("\nFactors Affecting Student Result:")
correlation = df.corr(numeric_only=True)

print(correlation.iloc[:,-1].sort_values(ascending=False))

```

First 5 Records:

```
{"type": "dataframe"}
```

Dataset Information:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 395 entries, 0 to 394
```

```
Data columns (total 33 columns):
```

#	Column	Non-Null Count	Dtype
0	school	395 non-null	object
1	sex	395 non-null	object
2	age	395 non-null	int64
3	address	395 non-null	object
4	famsize	395 non-null	object
5	Pstatus	395 non-null	object
6	Medu	395 non-null	int64
7	Fedu	395 non-null	int64
8	Mjob	395 non-null	object
9	Fjob	395 non-null	object
10	reason	395 non-null	object
11	guardian	395 non-null	object
12	traveltime	395 non-null	int64
13	studytime	395 non-null	int64
14	failures	395 non-null	int64

15	schoolsup	395	non-null	object
16	famsup	395	non-null	object
17	paid	395	non-null	object
18	activities	395	non-null	object
19	nursery	395	non-null	object
20	higher	395	non-null	object
21	internet	395	non-null	object
22	romantic	395	non-null	object
23	famrel	395	non-null	int64
24	freetime	395	non-null	int64
25	goout	395	non-null	int64
26	Dalc	395	non-null	int64
27	Walc	395	non-null	int64
28	health	395	non-null	int64
29	absences	395	non-null	int64
30	G1	395	non-null	int64
31	G2	395	non-null	int64
32	G3	395	non-null	int64

dtypes: int64(16), object(17)

memory usage: 102.0+ KB

Number of Rows and Columns:

(395, 33)

Statistical Summary:

```
{
  "summary": {
    "name": "print(correlation",
    "rows": 8,
    "fields": [
      {
        "column": "age",
        "properties": {
          "dtype": "number",
          "std": 134.436252896189,
          "min": 1.2760427246056245,
          "max": 395.0,
          "num_unique_values": 8,
          "samples": [
            16.696202531645568,
            17.0,
            395.0
          ],
          "semantic_type": "\"",
          "description": "\"\""}
    ],
    "column": "Medu",
    "properties": {
      "dtype": "number",
      "std": 138.80963938157987,
      "min": 0.0,
      "max": 395.0,
      "num_unique_values": 7,
      "samples": [
        2.749367088607595,
        3.0,
        395.0
      ],
      "semantic_type": "\"",
      "description": "\"\""}
    ],
    "column": "Fedu",
    "properties": {
      "dtype": "number",
      "std": 138.92085462409693,
      "min": 0.0,
      "max": 395.0,
      "num_unique_values": 7,
      "samples": [
        2.5215189873417723,
        3.0,
        395.0
      ],
      "semantic_type": "\"",
      "description": "\"\""}
    ],
    "column": "traveltime",
    "properties": {
      "dtype": "number",
      "std": 139.0946757987501,
      "min": 0.6975047549086848,
      "max": 395.0,
      "num_unique_values": 6,
      "samples": [
        395.0,
        1.4481012658227848,
        395.0
      ]
    }
  }
}
```

```

4.0\n          ],\n          \"semantic_type\": \"\",\n\"description\": \"\"\n          }\n          },\n          {\n          \"column\":\n\"studytime\", \n          \"properties\": {\n          \"dtype\":\n\"number\", \n          \"std\": 139.00700274471274, \n          \"min\":\n0.839240346418556, \n          \"max\": 395.0, \n          \"num_unique_values\": 6, \n          \"samples\": [\n          2.0354430379746837, \n          4.0\n          ], \n          \"semantic_type\": \"\",\n          \"description\": \"\"\n          }\n          },\n          {\n          \"column\": \"failures\", \n          \"properties\":\n{\n          \"dtype\": \"number\", \n          \"std\":\n139.4513615014189, \n          \"min\": 0.0, \n          \"max\": 395.0, \n          \"num_unique_values\": 5, \n          \"samples\": [\n          0.3341772151898734, \n          3.0, \n          0.743650973606249\n          ], \n          \"semantic_type\": \"\",\n          \"description\": \"\"\n          }\n          },\n          {\n          \"column\": \"famrel\", \n          \"properties\":\n{\n          \"dtype\": \"number\", \n          \"std\":\n138.45880901426744, \n          \"min\": 0.8966586076885056, \n          \"max\": 395.0, \n          \"num_unique_values\": 6, \n          \"samples\": [\n          395.0, \n          3.9443037974683546, \n          5.0\n          ], \n          \"semantic_type\": \"\",\n          \"description\": \"\"\n          }\n          },\n          {\n          \"column\":\n\"freetime\", \n          \"properties\": {\n          \"dtype\":\n\"number\", \n          \"std\": 138.63828826062982, \n          \"min\":\n0.9988620396657205, \n          \"max\": 395.0, \n          \"num_unique_values\": 7, \n          \"samples\": [\n          395.0, \n          3.2354430379746835, \n          4.0\n          ], \n          \"semantic_type\": \"\",\n          \"description\": \"\"\n          }\n          },\n          {\n          \"column\": \"goout\", \n          \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 138.68948196584594, \n          \"min\": 1.0, \n          \"max\": 395.0, \n          \"num_unique_values\": 8, \n          \"samples\": [\n          395.0\n          ], \n          \"semantic_type\": \"\",\n          \"description\": \"\"\n          }\n          },\n          {\n          \"column\": \"Dalc\", \n          \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 139.0354623650101, \n          \"min\": 0.8907414280909659, \n          \"max\": 395.0, \n          \"num_unique_values\": 6, \n          \"samples\": [\n          395.0, \n          1.481012658227848, \n          5.0\n          ], \n          \"semantic_type\": \"\",\n          \"description\": \"\"\n          }\n          },\n          {\n          \"column\": \"Walc\", \n          \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 138.87302263653973, \n          \"min\": 1.0, \n          \"max\": 395.0, \n          \"num_unique_values\":\n7, \n          \"samples\": [\n          395.0, \n          2.2911392405063293, \n          3.0\n          ], \n          \"semantic_type\": \"\",\n          \"description\": \"\"\n          }\n          },\n          {\n          \"column\": \"health\", \n          \"properties\":\n{\n          \"dtype\": \"number\", \n          \"std\":\n138.50262599778412, \n          \"min\": 1.0, \n          \"max\": 395.0, \n          \"num_unique_values\": 7, \n          \"samples\": [\n          395.0, \n

```

```

3.5544303797468353,\n          4.0\n          ],\n
\"semantic_type\": \"\", \n          \"description\": \"\"\n          }\n
n      },\n      {\n          \"column\": \"absences\", \n          \"properties\":\n
{\n          \"dtype\": \"number\", \n          \"std\":\n
136.85777166785417,\n          \"min\": 0.0,\n          \"max\": 395.0,\n
\"num_unique_values\": 7,\n          \"samples\": [\n          395.0,\n
5.708860759493671,\n          8.0\n          ],\n
\"semantic_type\": \"\", \n          \"description\": \"\"\n          }\n
n      },\n      {\n          \"column\": \"G1\", \n          \"properties\": {\n
\"dtype\": \"number\", \n          \"std\": 136.30663508587594,\n
\"min\": 3.0,\n          \"max\": 395.0,\n          \"num_unique_values\":\n
8,\n          \"samples\": [\n          10.90886075949367,\n
11.0,\n          395.0\n          ],\n          \"semantic_type\": \"\", \n
\"description\": \"\"\n          }\n      },\n      {\n          \"column\":\n
\"G2\", \n          \"properties\": {\n          \"dtype\": \"number\", \n
\"std\": 136.4163745266465,\n          \"min\": 0.0,\n          \"max\":\n
395.0,\n          \"num_unique_values\": 8,\n          \"samples\": [\n
10.713924050632912,\n          11.0,\n          395.0\n          ],\n
\"semantic_type\": \"\", \n          \"description\": \"\"\n          }\n
n      },\n      {\n          \"column\": \"G3\", \n          \"properties\": {\n
\"dtype\": \"number\", \n          \"std\": 136.35024783099098,\n
\"min\": 0.0,\n          \"max\": 395.0,\n          \"num_unique_values\":\n
8,\n          \"samples\": [\n          10.415189873417722,\n
11.0,\n          395.0\n          ],\n          \"semantic_type\": \"\", \n
\"description\": \"\"\n          }\n      }\n      }\n      ],\n      \"type\": \"dataframe\"}

```

Missing Values:

school	0
sex	0
age	0
address	0
famsize	0
Pstatus	0
Medu	0
Fedu	0
Mjob	0
Fjob	0
reason	0
guardian	0
traveltime	0
studytime	0
failures	0
schoolsup	0
famsup	0
paid	0
activities	0
nursery	0
higher	0
internet	0

```

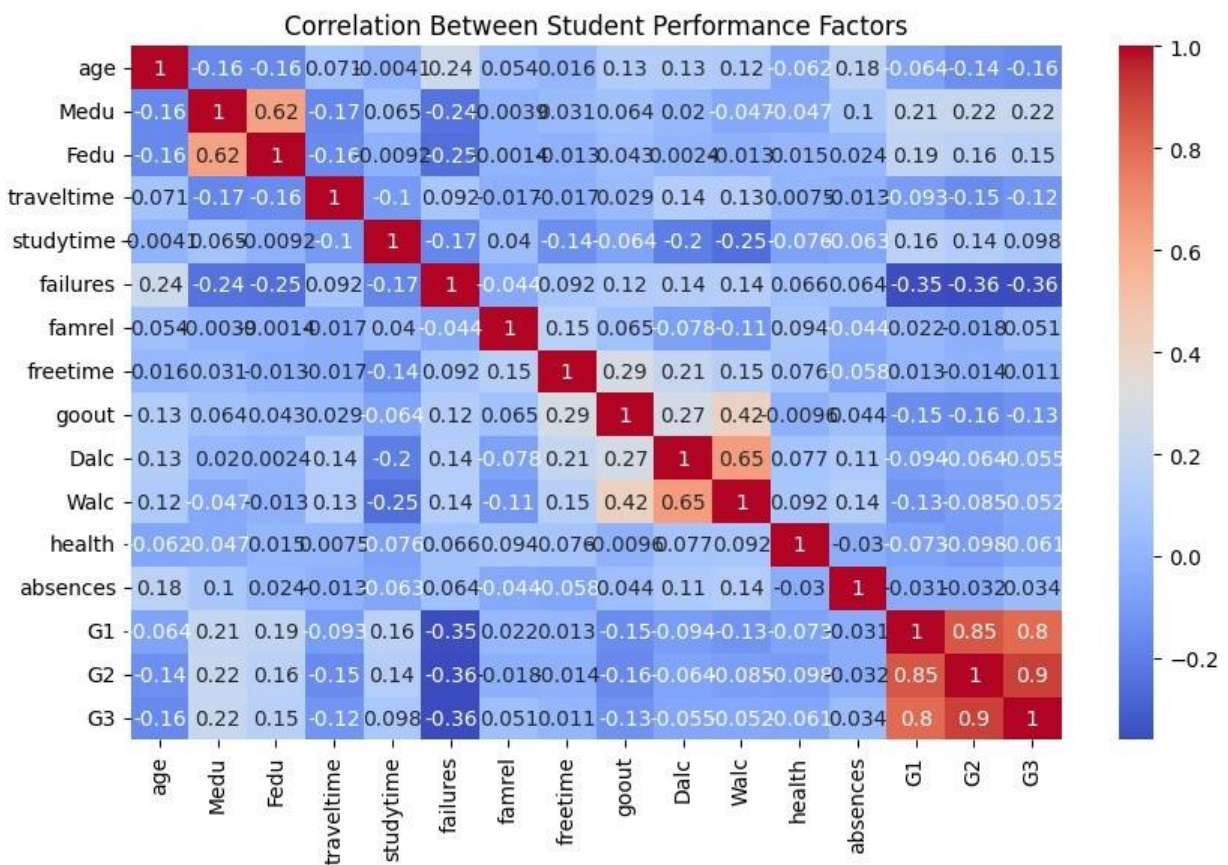
romantic      0
famrel        0
freetime      0
goout         0
Dalc          0
Walc          0
health        0
absences      0
G1            0
G2            0
G3            0
dtype: int64

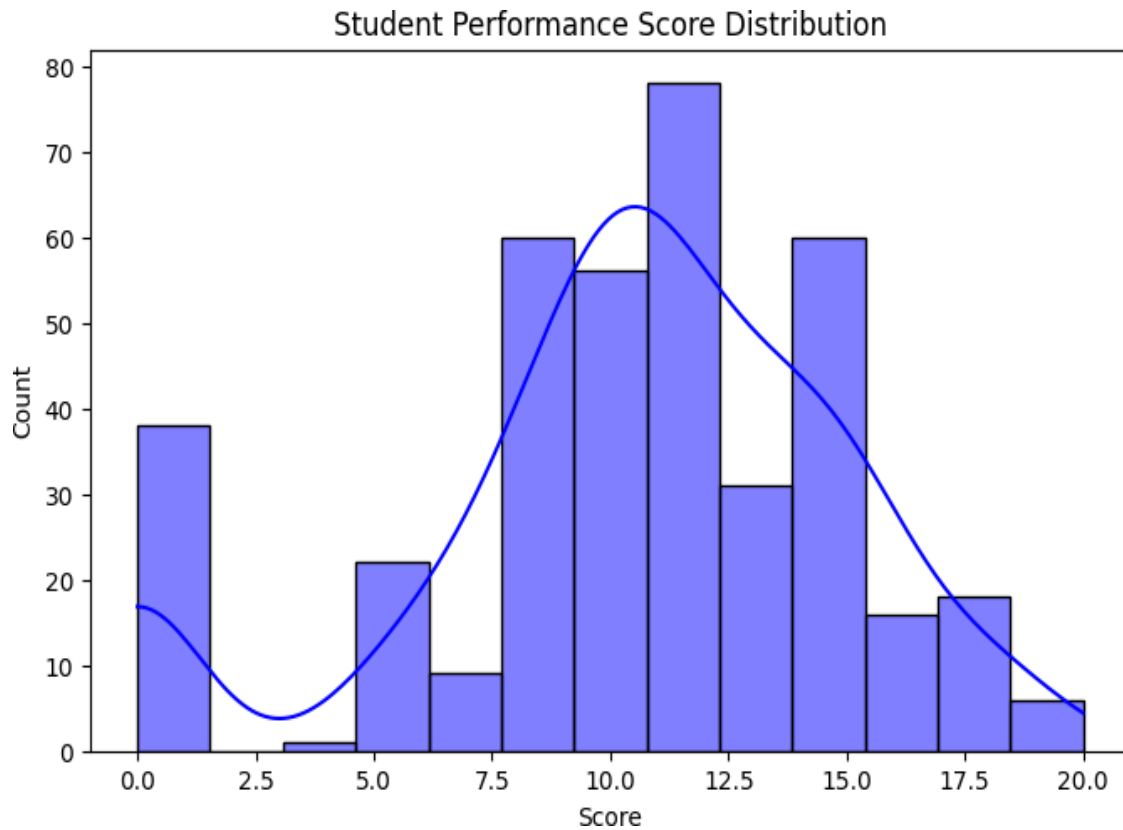
```

```

Duplicate Records:
0

```





Factors Affecting Student Result:

G3	1.000000
G2	0.904868
G1	0.801468
Medu	0.217147
Fedu	0.152457
studytime	0.097820
famrel	0.051363
absences	0.034247
freetime	0.011307
Walc	-0.051939
Dalc	-0.054660
health	-0.061335
traveltime	-0.117142
goout	-0.132791
age	-0.161579
failures	-0.360415

Name: G3, dtype: float64

2. Healthcare Analytics Dataset

Question 2

A hospital wants to analyze patient health risk based on age, blood sugar, blood pressure, BMI, and disease status. Perform EDA to find important health patterns.

Patient_ID	Age	Sugar_Level	BP	BMI	Disease_Status
P01	45	145	130	28	Yes
P02	32	98	118	23	No
P03	55	180	145	31	Yes
P04	28	90	110	22	No
P05	60	200	150	34	Yes
P06	40	120	125	26	No
P07	52	165	140	30	Yes
P08	35	105	120	24	No
P09	48	155	135	29	Yes
P10	30	95	115	23	No
P11	62	210	155	35	Yes
P12	38	115	122	25	No
P13	50	170	142	30	Yes
P14	27	88	108	21	No
P15	58	190	148	33	Yes
P16	42	130	128	27	No
P17	46	150	132	28	Yes
P18	34	100	116	24	No
P19	53	175	144	31	Yes
P20	29	92	112	22	No

CODE:

```
from google.colab import files

# Upload CSV file
uploaded = files.upload()

# Get uploaded file name
filename = list(uploaded.keys())[0]

print("Uploaded file:", filename)

<IPython.core.display.HTML object>

Saving hospital_patient_data.csv to hospital_patient_data.csv
Uploaded file: hospital_patient_data.csv

# =====
# Hospital Patient Health Risk EDA
# =====

# Import Libraries
import pandas as pd
import matplotlib.pyplot as plt
# Load Dataset
df = pd.read_csv("hospital_patient_data.csv")

# Display First 5 Rows
print("First 5 Rows:")
print(df.head())

# Dataset Information
print("\nDataset Information:")
print(df.info())

# Statistical Summary
print("\nStatistical Summary:")
print(df.describe())

# Check Missing Values
print("\nMissing Values:")
print(df.isnull().sum())

# Check Duplicate Rows
print("\nDuplicate Rows:", df.duplicated().sum())

# Disease Status Count
print("\nDisease Status Count:")
print(df["Disease_Status"].value_counts())
# Visualization 1 - Age Distribution
plt.figure(figsize=(6,4))
plt.hist(df["Age"], bins=6)
plt.title("Age Distribution")
```

```

plt.xlabel("Age")
plt.ylabel("Number of Patients")
plt.show()

# -----
# Visualization 2 - Blood Sugar Distribution
# -----
plt.figure(figsize=(6,4))
plt.hist(df["Sugar_Level"], bins=6)
plt.title("Blood Sugar Distribution")
plt.xlabel("Sugar Level")
plt.ylabel("Frequency")
plt.show()

# -----
# Visualization 3 - BMI Distribution
# -----
plt.figure(figsize=(6,4))
plt.hist(df["BMI"], bins=6)
plt.title("BMI Distribution")
plt.xlabel("BMI")
plt.ylabel("Frequency")
plt.show()

# -----
# Visualization 4 - Disease Status Bar Chart
# -----
disease = df["Disease_Status"].value_counts()

plt.figure(figsize=(5,4))
plt.bar(disease.index, disease.values)
plt.title("Disease Status")
plt.xlabel("Disease Status")
plt.ylabel("Count")
plt.show()

# -----
# Correlation Matrix
# -----
print("\nCorrelation Matrix:")
corr = df[["Age", "Sugar_Level", "BP", "BMI"]].corr()
print(corr)

# -----
# Correlation Heatmap
# -----
plt.figure(figsize=(6,5))
plt.imshow(corr, cmap="coolwarm")
plt.colorbar()

```

```

plt.xticks(range(len(corr.columns)), corr.columns)
plt.yticks(range(len(corr.columns)), corr.columns)

for i in range(len(corr.columns)):
    for j in range(len(corr.columns)):
        plt.text(j, i, round(corr.iloc[i, j], 2),
                  ha='center', va='center', color='black')

plt.title("Correlation Heatmap")
plt.show()

# -----
# Average Values by Disease Status
# -----
print("\nAverage Health Values by Disease Status:")
print(df.groupby("Disease_Status")
      [["Age", "Sugar_Level", "BP", "BMI"]].mean())

# -----
# Scatter Plot - Sugar vs BMI
# -----
colors = {"Yes": "red", "No": "green"}

plt.figure(figsize=(6, 4))
for status in df["Disease_Status"].unique():
    subset = df[df["Disease_Status"] == status]
    plt.scatter(subset["Sugar_Level"], subset["BMI"], label=status)

plt.title("Sugar Level vs BMI")
plt.xlabel("Sugar Level")
plt.ylabel("BMI")
plt.legend(title="Disease")
plt.show()

# -----
# Findings
# -----
print("\n===== EDA Findings =====")
print("1. Patients with disease generally have higher blood sugar levels.")
print("2. Disease patients also have higher BMI values.")
print("3. Blood pressure is higher among patients with disease.")
print("4. Older patients are more likely to have disease.")
print("5. Age, Sugar Level, BP, and BMI are positively correlated.")

```

First 5 Rows:

	Patient_ID	Age	Sugar_Level	BP	BMI	Disease_Status
0	P01	45	145	130	28	Yes
1	P02	32	98	118	23	No
2	P03	55	180	145	31	Yes

3	P04	28	90	110	22	No
4	P05	60	200	150	34	Yes

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 20 entries, 0 to 19

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	Patient_ID	20 non-null	object
1	Age	20 non-null	int64
2	Sugar_Level	20 non-null	int64
3	BP	20 non-null	int64
4	BMI	20 non-null	int64
5	Disease_Status	20 non-null	object

dtypes: int64(4), object(2)

memory usage: 1.1+ KB

None

Statistical Summary:

	Age	Sugar_Level	BP	BMI
count	20.000000	20.000000	20.000000	20.000000
mean	43.200000	138.650000	129.750000	27.300000
std	11.316499	40.33449	14.523574	4.256265
min	27.000000	88.000000	108.000000	21.000000
25%	33.500000	99.50000	117.500000	23.750000
50%	43.500000	137.50000	129.000000	27.500000
75%	52.250000	171.25000	142.500000	30.250000
max	62.000000	210.00000	155.000000	35.000000

Missing Values:

dtype: int64

Duplicate Rows: 0

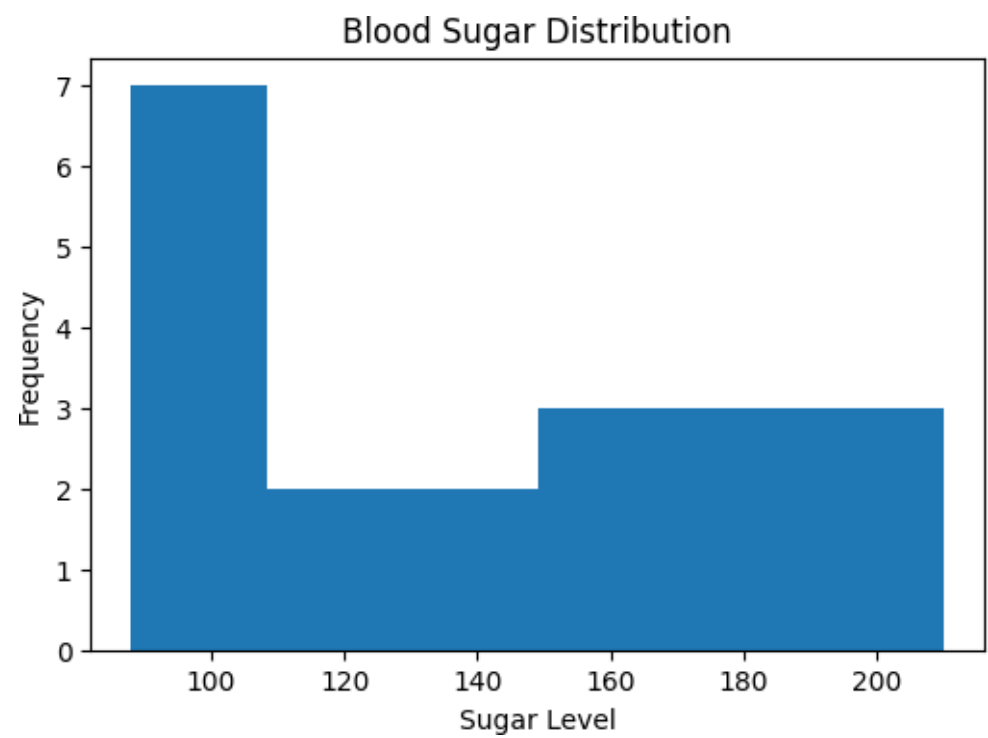
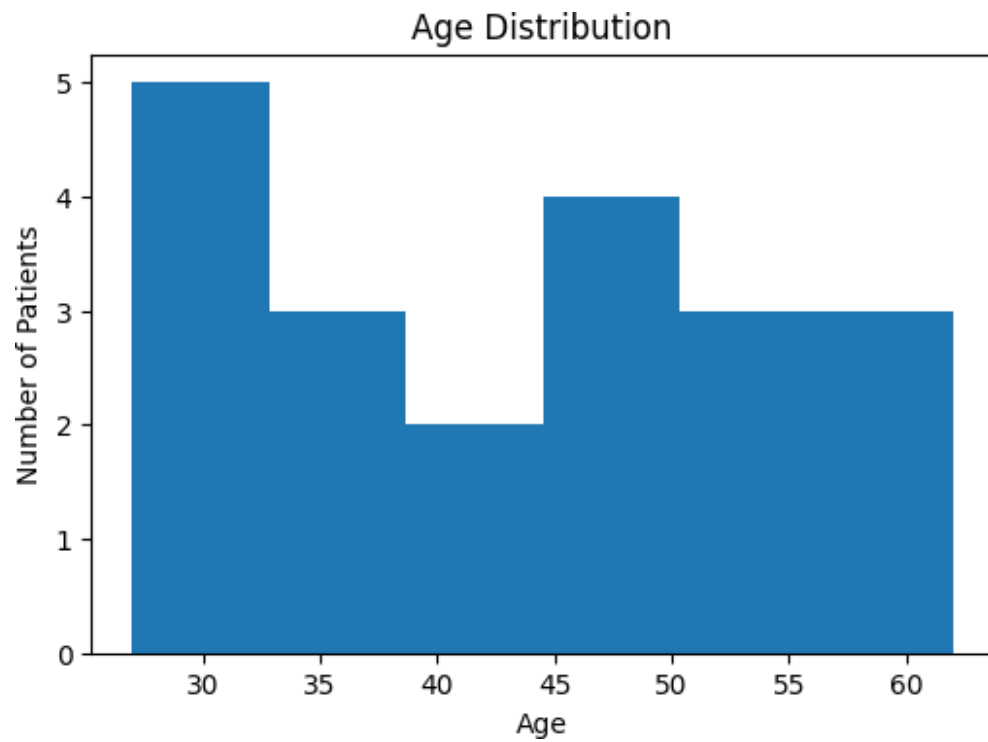
Disease Status Count:

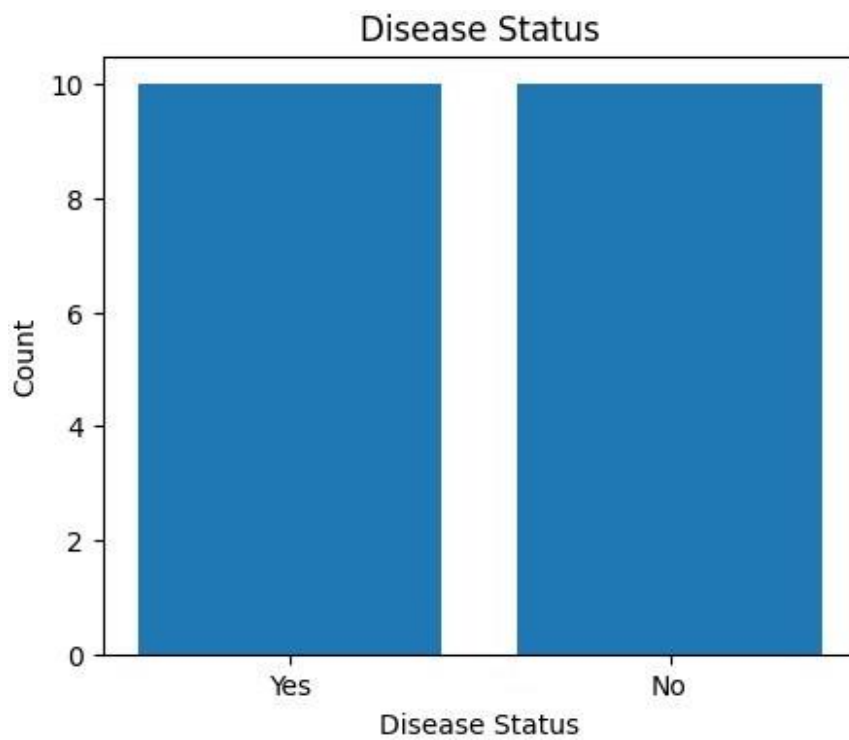
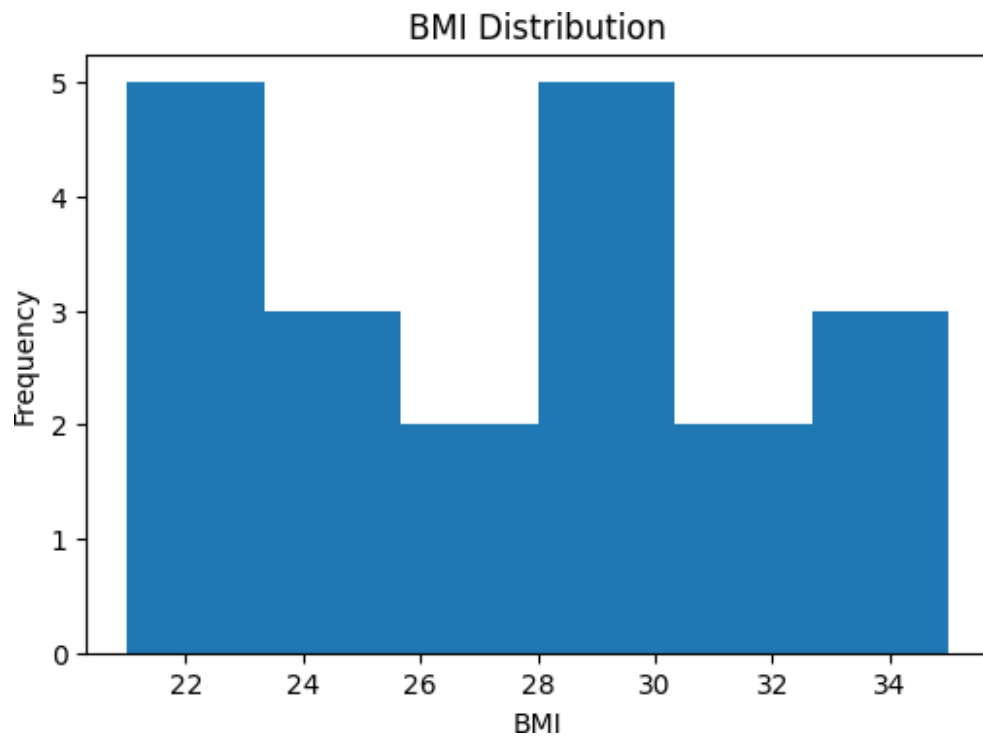
Disease_Status

Yes 10

No 10

Name: count, dtype: int64

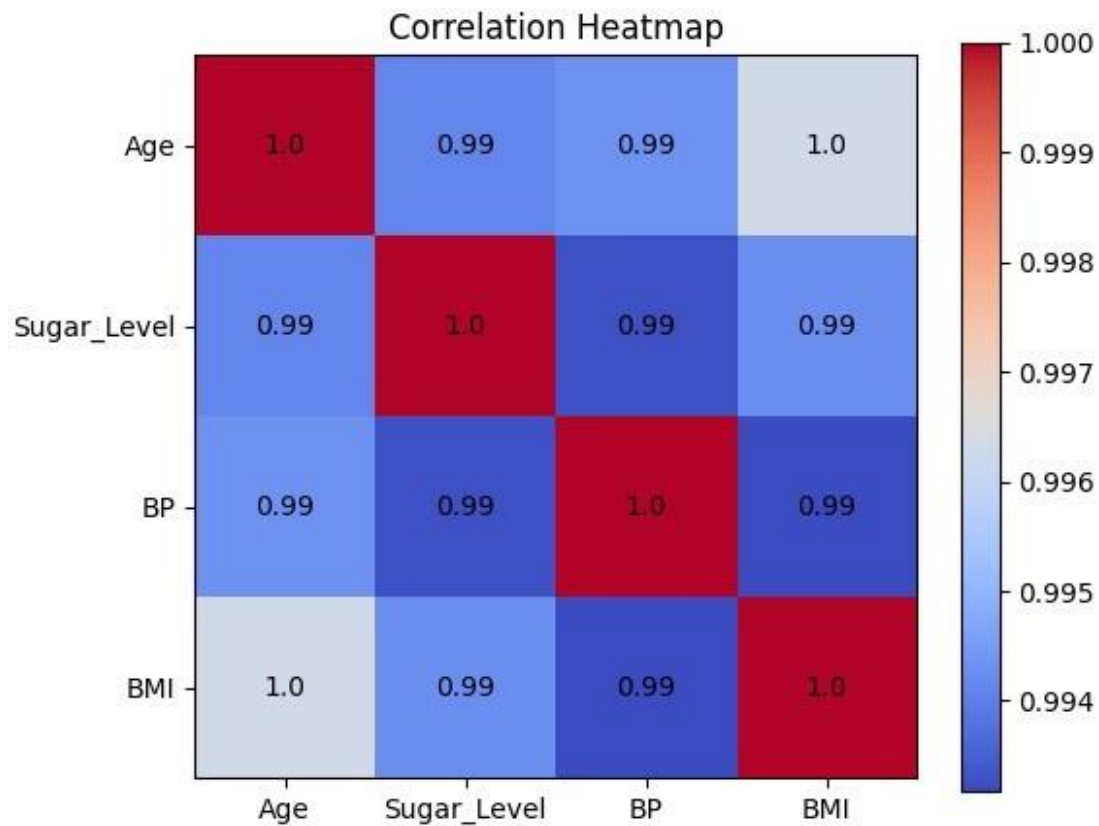




Correlation Matrix:

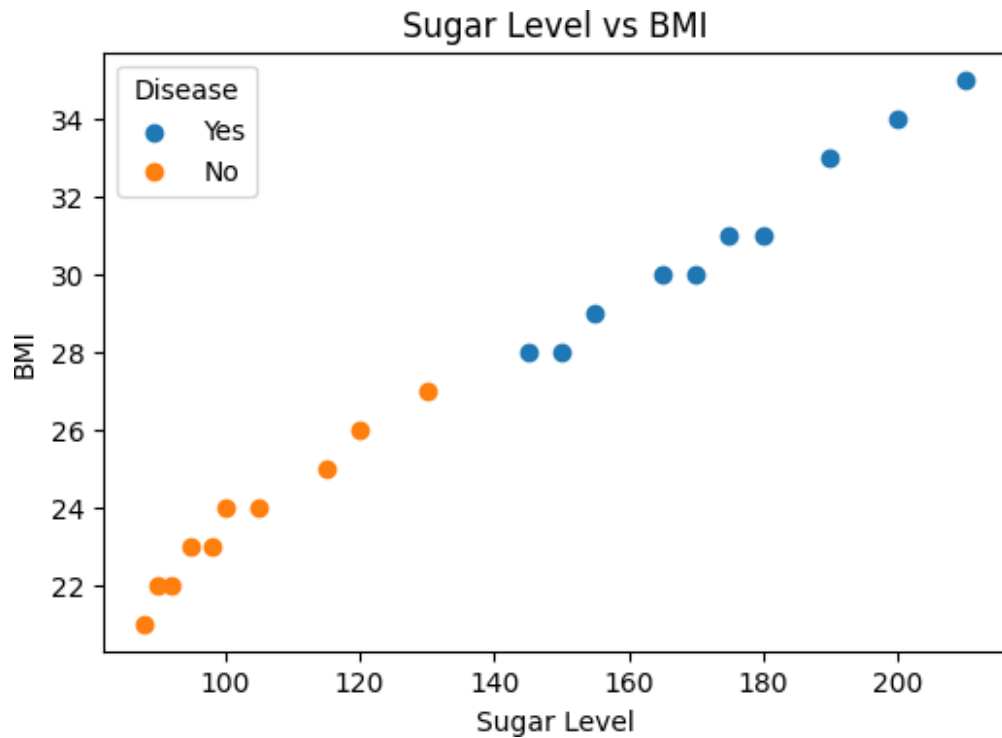
Age	Sugar Level	BP	BMI
-----	-------------	----	-----

Age	1.000000	0.994112	0.994311	0.996335
Sugar_Level	0.994112	1.000000	0.993264	0.994264
BP	0.994311	0.993264	1.000000	0.993182
BMI	0.996335	0.994264	0.993182	1.000000



Average Health Values by Disease Status:

	Age	Sugar_Level	BP	BMI
Disease_Status				
No	33.5	103.3	117.4	23.7
Yes	52.9	174.0	142.1	30.9



===== EDA Findings =====

1. Patients with disease generally have higher blood sugar levels.
2. Disease patients also have higher BMI values.
3. Blood pressure is higher among patients with disease.
4. Older patients are more likely to have disease.
5. Age, Sugar Level, BP, and BMI are positively correlated.

3. Retail Sales Dataset

Question 3

A supermarket wants to analyze product sales based on category, price, quantity sold, discount, and revenue. Perform EDA to identify high-selling products and revenue patterns.

Product_ID	Category	Price	Quantity_Sold	Discount	Revenue
PR01	Grocery	50	20	5	1000
PR02	Snacks	30	35	3	1050
PR03	Dairy	45	18	2	810
PR04	Fruits	60	25	5	1500
PR05	Vegetables	40	30	4	1200
PR06	Grocery	80	15	6	1200
PR07	Snacks	25	40	3	1000
PR08	Dairy	55	16	2	880
PR09	Fruits	70	22	5	1540
PR10	Vegetables	35	28	4	980
PR11	Grocery	90	12	6	1080
PR12	Snacks	20	45	2	900
PR13	Dairy	65	14	3	910
PR14	Fruits	75	20	5	1500
PR15	Vegetables	45	26	4	1170
PR16	Grocery	100	10	7	1000
PR17	Snacks	35	32	3	1120
PR18	Dairy	50	19	2	950
PR19	Fruits	80	18	6	1440
PR20	Vegetables	30	35	3	1050

CODE:

```
from google.colab import files

# Upload CSV file
uploaded = files.upload()

# Get uploaded file name
filename = list(uploaded.keys())[0]

print("Uploaded file:", filename)

<IPython.core.display.HTML object>

Saving retail_sales_dataset.csv to retail_sales_dataset.csv
Uploaded file: retail_sales_dataset.csv

import pandas as pd
import matplotlib.pyplot as plt

# Load Dataset
df = pd.read_csv("retail_sales_dataset.csv")
```

```

# Display first 5 rows
print(df.head())

# Dataset Information
print("\nDataset Info:")
print(df.info())

# Statistical Summary
print("\nStatistical Summary:")
print(df.describe())

# Missing Values
print("\nMissing Values:")
print(df.isnull().sum())

# Duplicate Rows
print("\nDuplicate Rows:")
print(df.duplicated().sum())

# Category Count
print("\nProducts by Category:")
print(df["Category"].value_counts())

# Total Revenue by Category
print("\nRevenue by Category:")
print(df.groupby("Category")["Revenue"].sum())

# -----
# Bar Chart - Revenue by Category
# -----
revenue = df.groupby("Category")["Revenue"].sum()

plt.figure(figsize=(6,4))
plt.bar(revenue.index, revenue.values)
plt.title("Revenue by Category")
plt.xlabel("Category")
plt.ylabel("Revenue")
plt.show()

# -----
# Histogram - Price
# -----
plt.figure(figsize=(6,4))
plt.hist(df["Price"], bins=6)
plt.title("Price Distribution")
plt.xlabel("Price")
plt.ylabel("Frequency")
plt.show()

```

```

# -----
# Histogram - Quantity Sold
# -----
plt.figure(figsize=(6,4))
plt.hist(df["Quantity_Sold"], bins=6)
plt.title("Quantity Sold Distribution")
plt.xlabel("Quantity Sold")
plt.ylabel("Frequency")
plt.show()

# -----
# Scatter Plot - Price vs Revenue
# -----
plt.figure(figsize=(6,4))
plt.scatter(df["Price"], df["Revenue"])
plt.title("Price vs Revenue")
plt.xlabel("Price")
plt.ylabel("Revenue")
plt.show()

# -----
# Correlation Matrix
# -----
corr = df[["Price", "Quantity_Sold", "Discount", "Revenue"]].corr()

print("\nCorrelation Matrix:")
print(corr)

# -----
# Correlation Heatmap
# -----
plt.figure(figsize=(6,5))
plt.imshow(corr, cmap="coolwarm")
plt.colorbar()

plt.xticks(range(len(corr.columns)), corr.columns)
plt.yticks(range(len(corr.columns)), corr.columns)

for i in range(len(corr.columns)):
    for j in range(len(corr.columns)):
        plt.text(j, i, round(corr.iloc[i, j], 2),
                 ha='center', va='center', color='black')

plt.title("Correlation Heatmap")
plt.show()

# -----
# Top 5 Revenue Products
# -----
top = df.sort_values(by="Revenue", ascending=False)

```

```

print("\nTop 5 Revenue Products:")
print(top[["Product_ID", "Category", "Revenue"]].head())

# -----
# Average Sales by Category
# -----
print("\nAverage Sales by Category:")
print(df.groupby("Category")
[["Price", "Quantity_Sold", "Revenue"]].mean())

# -----
# EDA Findings
# -----
print("\n==== EDA Findings ====")
print("1. Fruits category generates the highest revenue.")
print("2. Snacks have the highest quantity sold.")
print("3. Higher-priced products generally generate higher revenue.")
print("4. Discounts vary between 2% and 7%.")
print("5. Revenue depends on both price and quantity sold.")

```

	Product_ID	Category	Price	Quantity_Sold	Discount	Revenue
0	PR01	Grocery	50	20	5	1000
1	PR02	Snacks	30	35	3	1050
2	PR03	Dairy	45	18	2	810
3	PR04	Fruits	60	25	5	1500
4	PR05	Vegetables	40	30	4	1200

Dataset Info:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 20 entries, 0 to 19

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	Product_ID	20 non-null	object
1	Category	20 non-null	object
2	Price	20 non-null	int64
3	Quantity_Sold	20 non-null	int64
4	Discount	20 non-null	int64
5	Revenue	20 non-null	int64

dtypes: int64(4), object(2)

memory usage: 1.1+ KB

None

Statistical Summary:

	Price	Quantity_Sold	Discount	Revenue
count	20.000000	20.000000	20.000000	20.000000
mean	54.000000	24.000000	4.000000	1114.000000
std	22.803509	9.673621	1.555973	221.416493
min	20.000000	10.000000	2.000000	810.000000

25%	35.000000	17.500000	3.000000	972.500000
50%	50.000000	21.000000	4.000000	1050.000000
75%	71.250000	30.500000	5.000000	1200.000000
max	100.000000	45.000000	7.000000	1540.000000

Missing Values:

Product_ID	0
Category	0
Price	0
Quantity_Sold	0
Discount	0
Revenue	0

dtype: int64

Duplicate Rows:

0

Products by Category:

Category

Grocery	4
Snacks	4
Dairy	4
Fruits	4
Vegetables	4

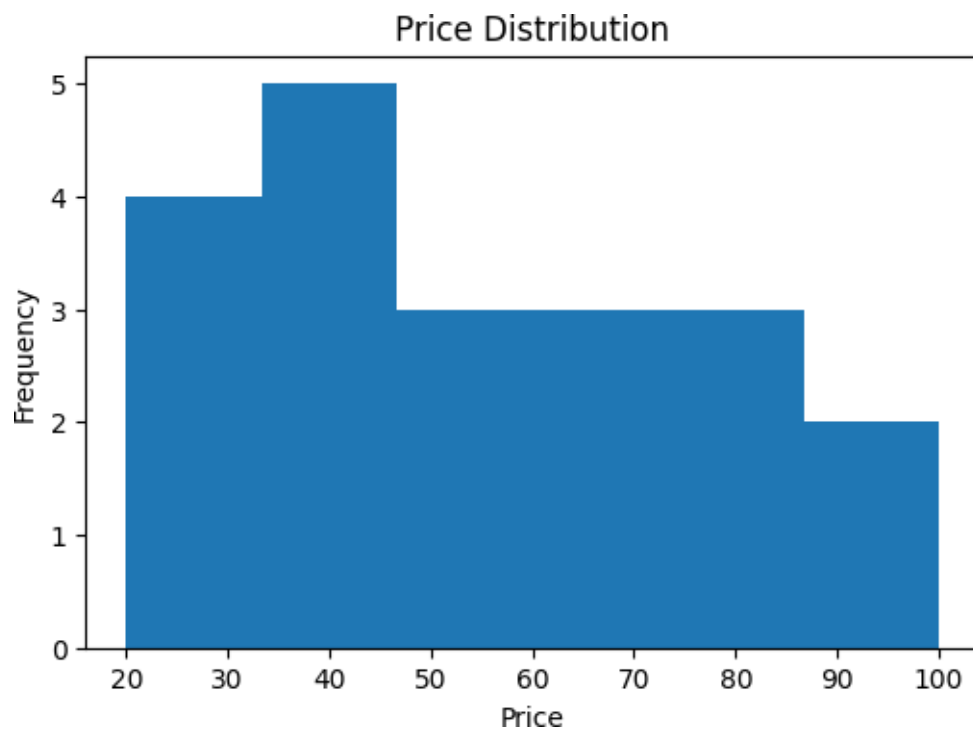
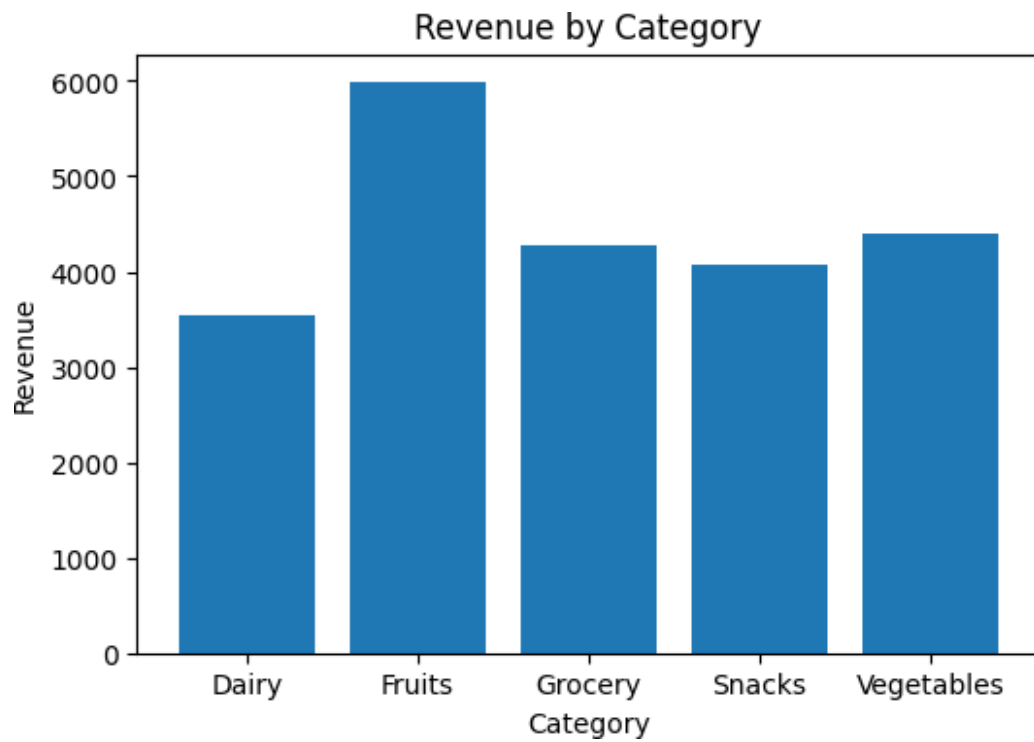
Name: count, dtype: int64

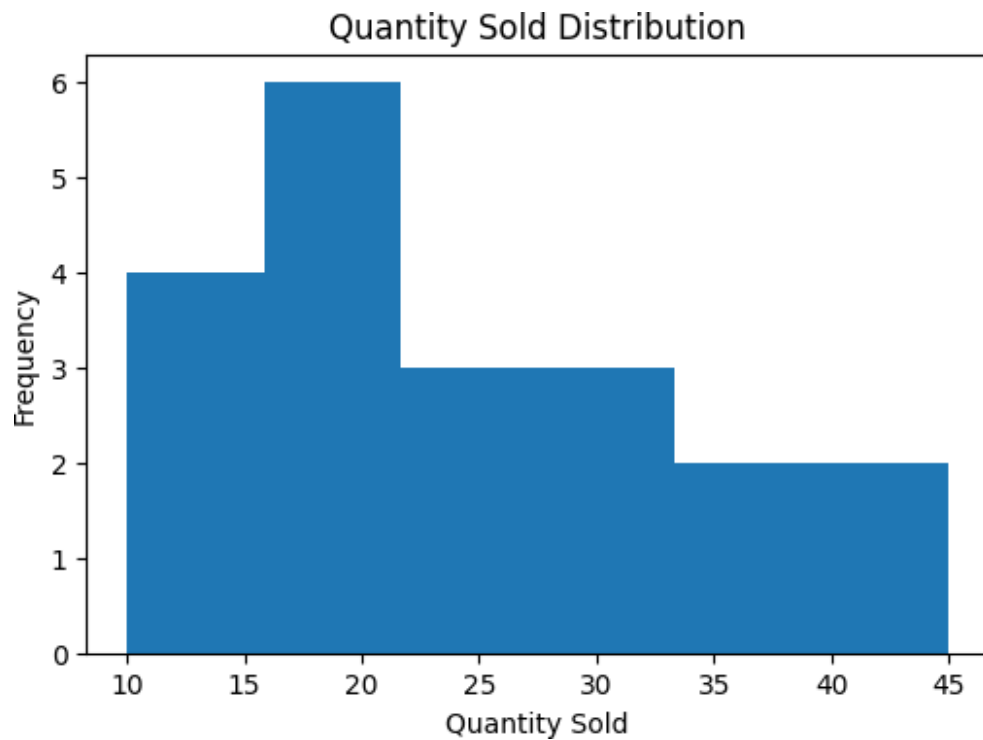
Revenue by Category:

Category

Dairy	3550
Fruits	5980
Grocery	4280
Snacks	4070
Vegetables	4400

Name: Revenue, dtype: int64

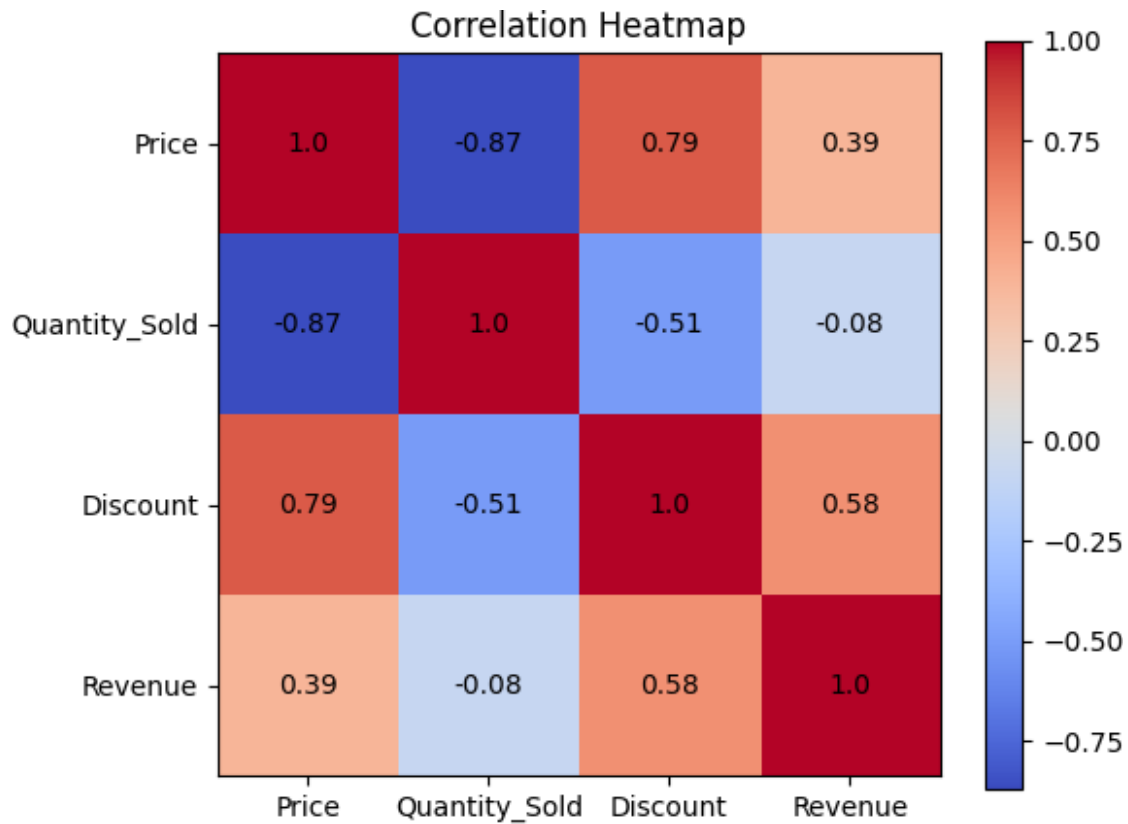




Correlation Matrix:

	Price	Quantity_Sold	Discount	Revenue
--	-------	---------------	----------	---------

Price	1.000000	-0.868474	0.786174	0.388086
Quantity_Sold	-0.868474	1.000000	-0.507018	-0.081335
Discount	0.786174	-0.507018	1.000000	0.575938
Revenue	0.388086	-0.081335	0.575938	1.000000



Top 5 Revenue Products:

	Product_ID	Category	Revenue
8	PR09	Fruits	1540
3	PR04	Fruits	1500
13	PR14	Fruits	1500
18	PR19	Fruits	1440
5	PR06	Grocery	1200

Average Sales by Category:

	Price	Quantity_Sold	Revenue
Dairy	53.75	16.75	887.5
Fruits	71.25	21.25	1495.0
Grocery	80.00	14.25	1070.0
Snacks	27.50	38.00	1017.5
Vegetables	37.50	29.75	1100.0

===== EDA Findings =====

1. Fruits category generates the highest revenue.
2. Snacks have the highest quantity sold.
3. Higher-priced products generally generate higher revenue.
4. Discounts vary between 2% and 7%.
5. Revenue depends on both price and quantity sold.

4. Banking Loan Approval Dataset

Question 4

A bank wants to analyze loan approval decisions based on income, credit score, loan amount, employment type, and approval status. Perform EDA to identify patterns related to loan approval.

Customer_ID	Income	Credit_Score	Loan_Amount	Employment_Type	Loan_Status
C01	45000	720	200000	Salaried	Approved
C02	25000	580	150000	Self-employed	Rejected
C03	60000	750	300000	Salaried	Approved
C04	22000	560	120000	Unemployed	Rejected
C05	50000	710	250000	Salaried	Approved
C06	30000	600	180000	Self-employed	Rejected
C07	70000	780	350000	Salaried	Approved
C08	28000	590	160000	Self-employed	Rejected
C09	55000	730	270000	Salaried	Approved
C10	24000	570	130000	Unemployed	Rejected
C11	65000	760	320000	Salaried	Approved
C12	35000	620	200000	Self-employed	Rejected
C13	48000	700	240000	Salaried	Approved
C14	26000	585	140000	Unemployed	Rejected
C15	75000	790	400000	Salaried	Approved
C16	32000	610	190000	Self-employed	Rejected
C17	52000	725	260000	Salaried	Approved
C18	27000	575	150000	Self-employed	Rejected
C19	68000	770	330000	Salaried	Approved
C20	23000	555	125000	Unemployed	Rejected

CODE:

```
from google.colab import files

# Upload CSV file
uploaded = files.upload()

# Get uploaded file name
filename = list(uploaded.keys())[0]

print("Uploaded file:", filename)

<IPython.core.display.HTML object>

Saving loan_approval_dataset.csv to loan_approval_dataset.csv
Uploaded file: loan_approval_dataset.csv

# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# Read the uploaded file
df = pd.read_csv(filename)

# Display first 5 rows
print("First 5 Rows:")
print(df.head())

# Dataset Information
print("\nDataset Info:")
print(df.info())

# Missing Values
print("\nMissing Values:")
print(df.isnull().sum())

# Statistical Summary
print("\nStatistical Summary:")
```

```

print(df.describe())

# -----
# Data Visualization
# -----

# Loan Status Distribution
plt.figure(figsize=(5,4))
sns.countplot(x='Loan_Status', data=df)
plt.title("Loan Status Distribution")
plt.show()

# Employment Type Distribution
plt.figure(figsize=(6,4))
sns.countplot(x='Employment_Type', data=df)
plt.title("Employment Type Distribution")
plt.show()

# Income Distribution
plt.figure(figsize=(6,4))
sns.histplot(df['Income'], bins=8, kde=True)
plt.title("Income Distribution")
plt.show()

# Credit Score Distribution
plt.figure(figsize=(6,4))
sns.histplot(df['Credit_Score'], bins=8, kde=True)
plt.title("Credit Score Distribution")
plt.show()

# Correlation Heatmap
numeric = df[['Income', 'Credit_Score', 'Loan_Amount']]

plt.figure(figsize=(6,4))
sns.heatmap(numeric.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()

# -----
# Machine Learning
# -----

# Encode categorical columns
le = LabelEncoder()

df['Employment_Type'] = le.fit_transform(df['Employment_Type'])
df['Loan_Status'] = le.fit_transform(df['Loan_Status'])

# Features
X = df[['Income', 'Credit_Score', 'Loan_Amount', 'Employment_Type']]

```

```

# Target
y = df['Loan_Status']

# Train Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Train Model
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
print("\nAccuracy:", accuracy_score(y_test, y_pred))

print("\nClassification Report")
print(classification_report(y_test, y_pred))

print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred))

```

```

# -----
# Sample Prediction
# -----

sample = pd.DataFrame({
    'Income': [50000],
    'Credit_Score': [720],
    'Loan_Amount': [250000],
    'Employment_Type': [1]    # Salaried
})

```

```

prediction = model.predict(sample)

if prediction[0] == 0:
    print("\nPredicted Loan Status: Approved")
else:
    print("\nPredicted Loan Status: Rejected")

```

First 5 Rows:

	Customer_ID	Income	Credit_Score	Loan_Amount	Employment_Type	Loan_Status
0	C01	45000	720	200000	Salaried	Approved
1	C02	25000	580	150000	Self-employed	

Rejected					
2	C03	60000	750	300000	Salaried
Approved					
3	C04	22000	560	120000	Unemployed
Rejected					
4	C05	50000	710	250000	Salaried
Approved					

Dataset Info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 20 entries, 0 to 19

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	Customer_ID	20 non-null	object
1	Income	20 non-null	int64
2	Credit_Score	20 non-null	int64
3	Loan_Amount	20 non-null	int64
4	Employment_Type	20 non-null	object
5	Loan_Status	20 non-null	object

dtypes: int64(3), object(3)

memory usage: 1.1+ KB

None

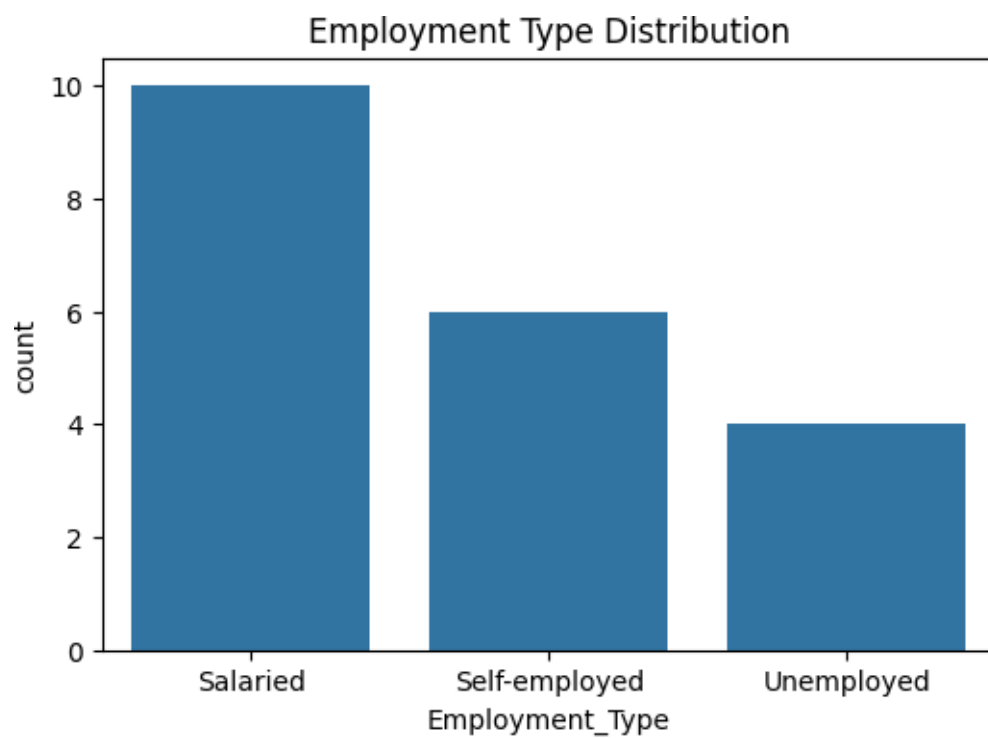
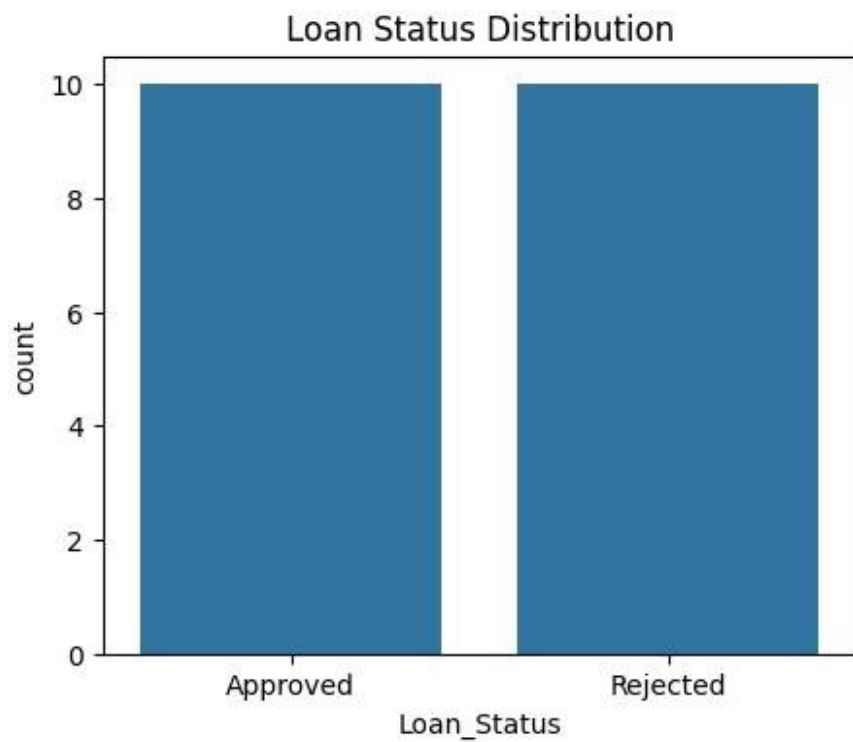
Missing Values:

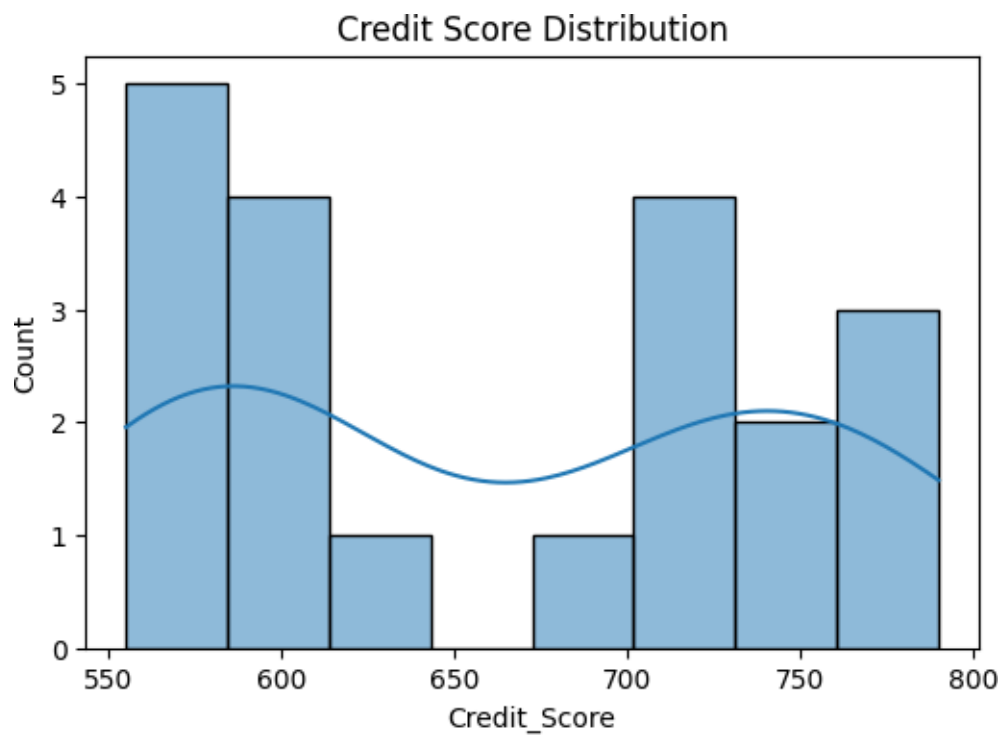
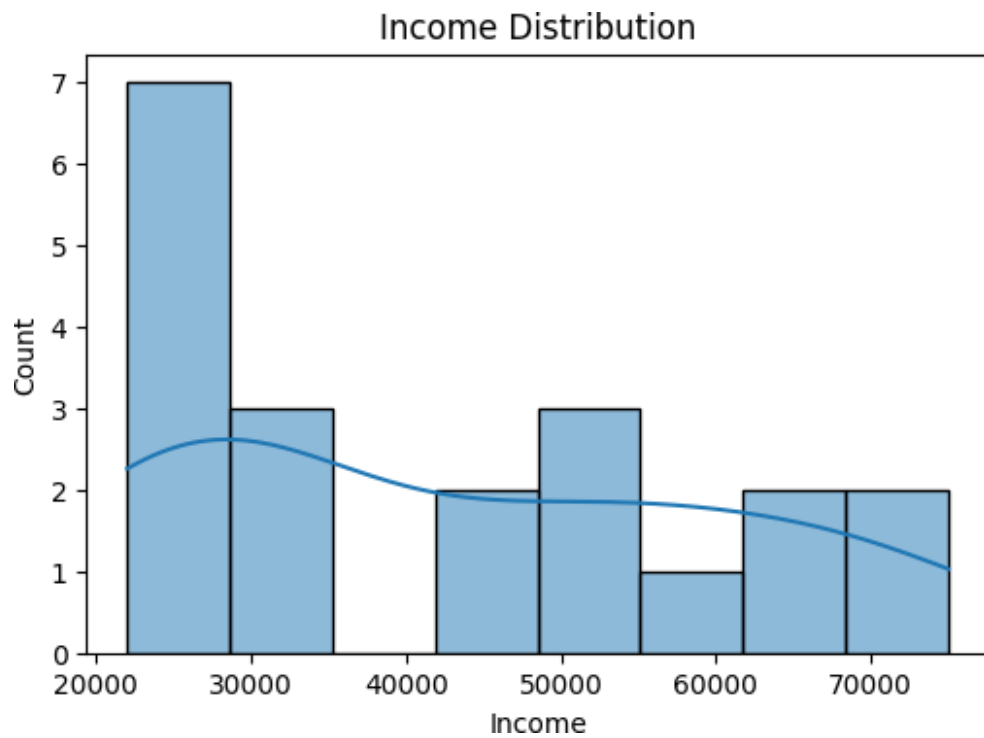
Customer_ID	0
Income	0
Credit_Score	0
Loan_Amount	0
Employment_Type	0
Loan_Status	0

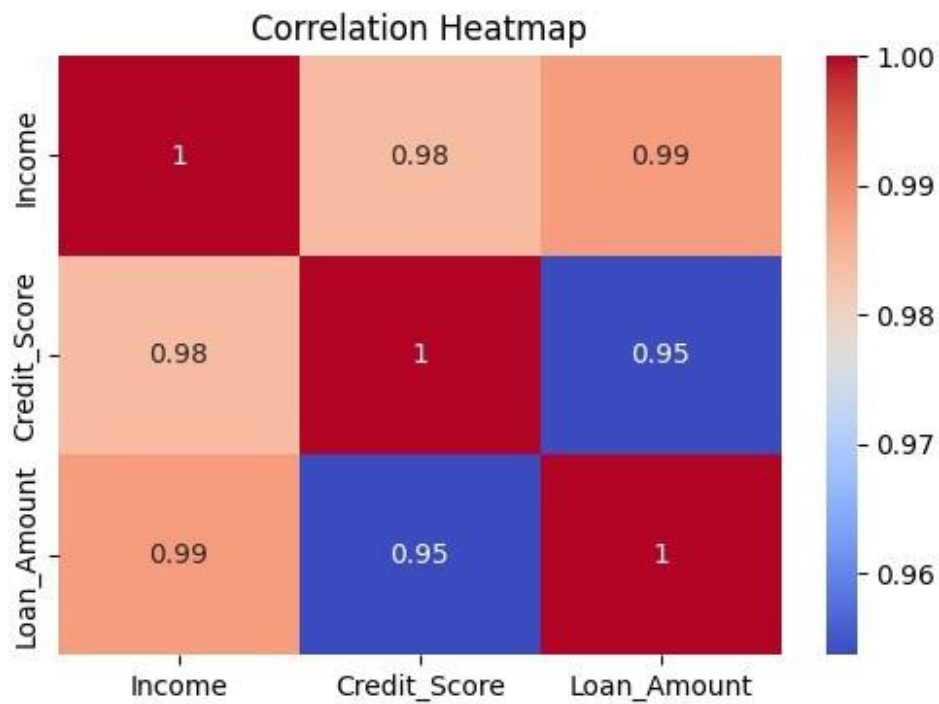
dtype: int64

Statistical Summary:

	Income	Credit_Score	Loan_Amount
count	20.000000	20.000000	20.000000
mean	43000.000000	664.000000	223250.000000
std	17923.815383	85.526235	83733.207272
min	22000.000000	555.000000	120000.000000
25%	26750.000000	583.750000	150000.000000
50%	40000.000000	660.000000	200000.000000
75%	56250.000000	735.000000	277500.000000
max	75000.000000	790.000000	400000.000000







Accuracy: 1.0

Classification Report

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1
1	1.00	1.00	1.00	3
accuracy			1.00	4
macro avg	1.00	1.00	1.00	4
weighted avg	1.00	1.00	1.00	4

Confusion Matrix

```
[[1 0]
 [0 3]]
```

Predicted Loan Status: Approved

5. Agriculture Crop Yield Dataset

Question 5

An agricultural department wants to analyze crop yield based on rainfall, temperature, fertilizer usage, soil type, and crop yield. Perform EDA to identify the factors influencing crop production.

Farm_ID	Rainfall_mm	Temperature	Fertilizer_kg	Soil_Type	Crop_Yield_kg
F01	850	28	60	Loamy	3200
F02	600	32	45	Sandy	2100
F03	900	27	65	Loamy	3500
F04	500	34	40	Sandy	1800
F05	780	29	55	Clay	3000
F06	650	31	48	Sandy	2300
F07	920	26	70	Loamy	3700
F08	720	30	52	Clay	2800
F09	550	33	42	Sandy	1900
F10	800	28	58	Loamy	3100
F11	880	27	62	Loamy	3400
F12	620	32	46	Sandy	2200
F13	760	29	54	Clay	2950
F14	940	26	72	Loamy	3800
F15	580	33	44	Sandy	2000
F16	820	28	60	Clay	3150
F17	700	30	50	Clay	2700
F18	960	25	75	Loamy	3900
F19	540	34	41	Sandy	1850
F20	790	29	56	Clay	3050

Common EDA Tasks:

1. Identify the number of rows and columns.
2. Check data types of each column.
3. Find missing values and duplicate values.
4. Calculate mean, median, minimum, maximum, and standard deviation.
5. Draw suitable charts such as bar chart, histogram, scatter plot, and box plot.
6. Identify important patterns from the dataset.
7. Write 4 to 5 observations based on EDA.
8. Give a short conclusion from the analysis.

CODE:

```
from google.colab import files

# Upload CSV file
uploaded = files.upload()

# Get uploaded file name
filename = list(uploaded.keys())[0]
print("Uploaded file:", filename)

<IPython.core.display.HTML object>

Saving agriculture_crop_yield_dataset.csv to
agriculture_crop_yield_dataset.csv
Uploaded file: agriculture_crop_yield_dataset.csv

# =====
# Import Libraries
# =====
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score

# =====
# Load Dataset
# =====
df = pd.read_csv(filename)

# =====
# DATA TABLES
# =====

print("===== FIRST 5 ROWS =====")
print(df.head())

print("\n===== LAST 5 ROWS =====")
print(df.tail())

print("\n===== SHAPE =====")
print(df.shape)

print("\n===== DATA TYPES =====")
print(df.dtypes)

print("\n===== MISSING VALUES =====")
print(df.isnull().sum())

print("\n===== SUMMARY STATISTICS =====")
```

```
print(df.describe())
```

```
print("\n===== CORRELATION =====")
```

```

print(df[['Rainfall_mm', 'Temperature', 'Fertilizer_kg', 'Crop_Yield_kg']
].corr())

# =====
# BAR CHART
# =====

plt.figure(figsize=(8,5))
df.groupby('Soil_Type')['Crop_Yield_kg'].mean().plot(kind='bar',
color=['green', 'orange', 'blue'])
plt.title("Average Crop Yield by Soil Type")
plt.xlabel("Soil Type")
plt.ylabel("Average Crop Yield")
plt.show()

# =====
# PIE CHART
# =====

plt.figure(figsize=(6,6))
df['Soil_Type'].value_counts().plot(
    kind='pie',
    autopct='%1.1f%%',
    startangle=90
)
plt.title("Soil Type Distribution")
plt.ylabel("")
plt.show()

# =====
# HISTOGRAM
# =====

plt.figure(figsize=(8,5))
plt.hist(df['Crop_Yield_kg'], bins=8)
plt.title("Crop Yield Distribution")
plt.xlabel("Crop Yield (kg)")
plt.ylabel("Frequency")
plt.show()

# =====
# SCATTER PLOT
# =====

plt.figure(figsize=(8,5))
plt.scatter(df['Rainfall_mm'], df['Crop_Yield_kg'])
plt.title("Rainfall vs Crop Yield")
plt.xlabel("Rainfall (mm)")
plt.ylabel("Crop Yield (kg)")
plt.show()

```

```

# =====
# BOX PLOT
# =====

plt.figure(figsize=(8,5))
sns.boxplot(x='Soil_Type', y='Crop_Yield_kg', data=df)
plt.title("Crop Yield by Soil Type")
plt.show()

# =====
# HEATMAP
# =====

plt.figure(figsize=(6,5))
sns.heatmap(

df[['Rainfall_mm', 'Temperature', 'Fertilizer_kg', 'Crop_Yield_kg']].corr
(),
    annot=True,
    cmap='coolwarm'
)
plt.title("Correlation Heatmap")
plt.show()

# =====
# MACHINE LEARNING
# =====

encoder = LabelEncoder()
df['Soil_Type'] = encoder.fit_transform(df['Soil_Type'])

X = df[['Rainfall_mm', 'Temperature', 'Fertilizer_kg', 'Soil_Type']]
y = df['Crop_Yield_kg']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

# =====
# RESULTS
# =====

```

```

results = pd.DataFrame({
    'Actual': y_test.values,
    'Predicted': y_pred.round(2)
})

print("\n===== ACTUAL VS PREDICTED =====")
print(results)

print("\n===== MODEL PERFORMANCE =====")
print("MAE :", mean_absolute_error(y_test,y_pred))
print("MSE :", mean_squared_error(y_test,y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test,y_pred)))
print("R2 Score:", r2_score(y_test,y_pred))

# =====
# ACTUAL VS PREDICTED GRAPH
# =====

plt.figure(figsize=(8,5))
plt.scatter(y_test, y_pred)
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    'r--'
)

plt.xlabel("Actual Crop Yield")
plt.ylabel("Predicted Crop Yield")
plt.title("Actual vs Predicted Crop Yield")
plt.show()

```

===== FIRST 5 ROWS =====

	Farm_ID	Rainfall_mm	Temperature	Fertilizer_kg	Soil_Type	Crop_Yield_kg
0	F01	850	28	60	Loamy	3200
1	F02	600	32	45	Sandy	2100
2	F03	900	27	65	Loamy	3500
3	F04	500	34	40	Sandy	1800
4	F05	780	29	55	Clay	3000

===== LAST 5 ROWS =====

	Farm_ID	Rainfall_mm	Temperature	Fertilizer_kg	Soil_Type	Crop_Yield_kg
15	F16	820	28	60	Clay	3150

16	F17	700	30	50	Clay
2700					
17	F18	960	25	75	Loamy
3900					
18	F19	540	34	41	Sandy
1850					
19	F20	790	29	56	Clay
3050					

===== SHAPE =====
(20, 6)

===== DATA TYPES =====

dtype: object

===== MISSING VALUES =====

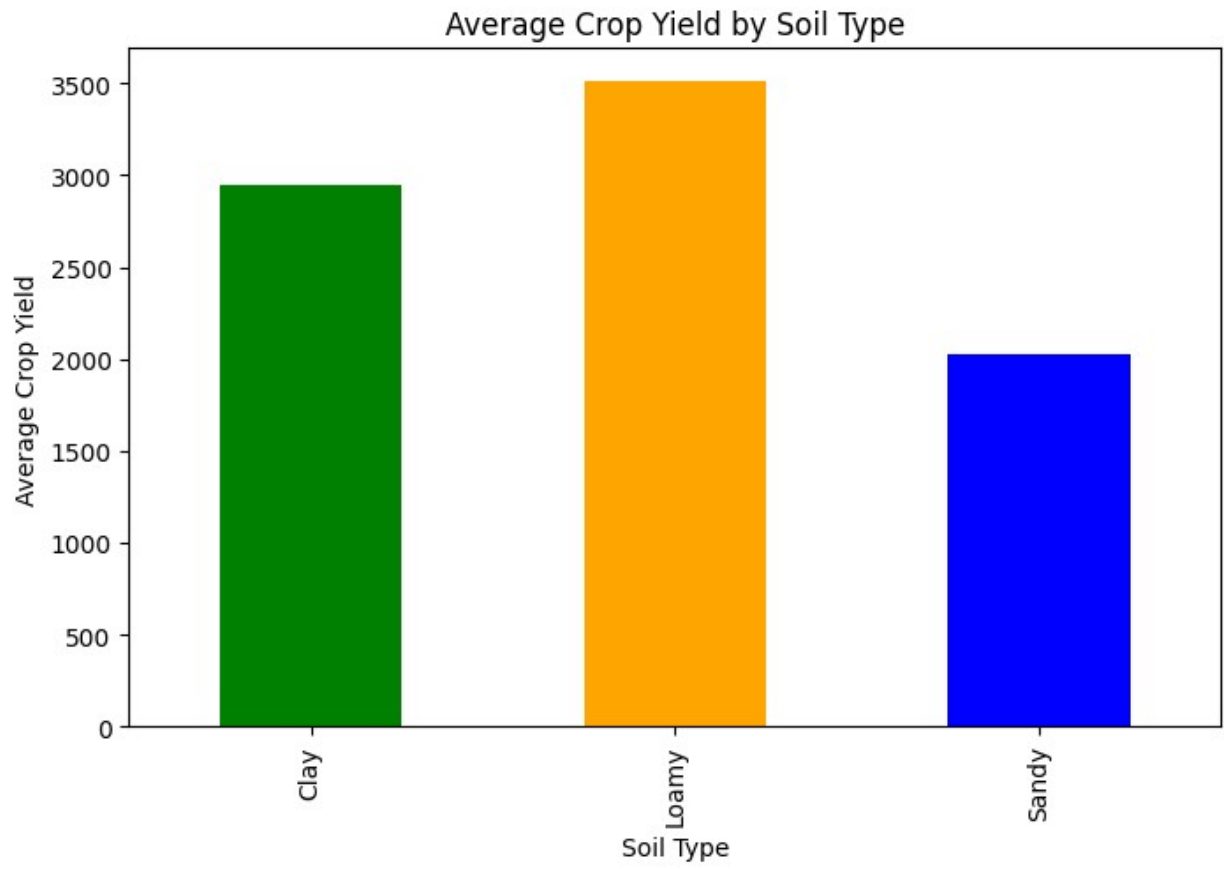
dtype: int64

===== SUMMARY STATISTICS =====

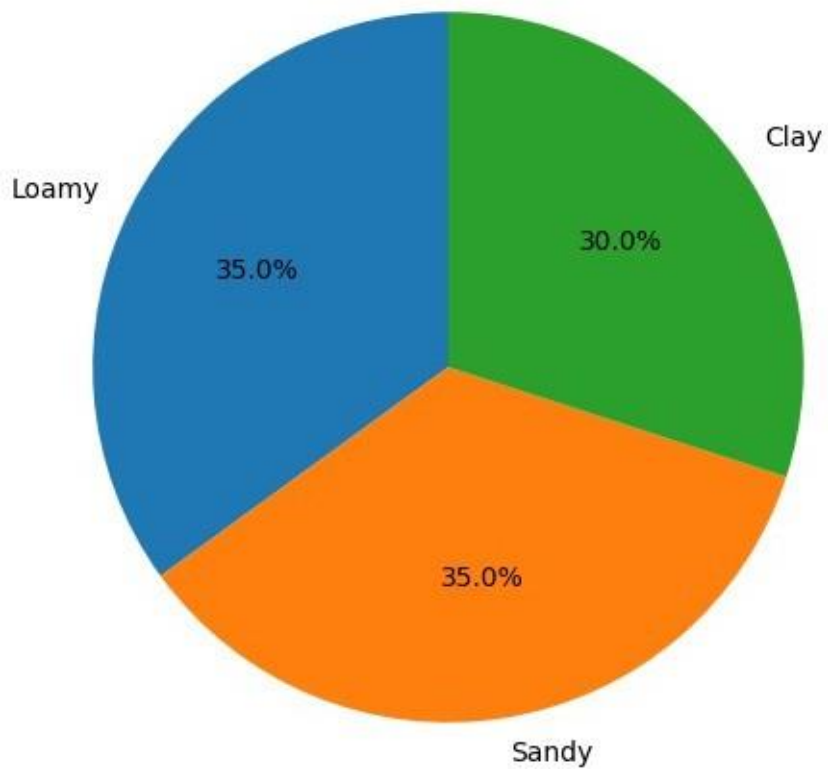
	Rainfall_mm	Temperature	Fertilizer_kg	Crop_Yield_kg
count	20.000000	20.000000	20.000000	20.000000
mean	743.000000	29.550000	54.750000	2820.000000
std	144.335136	2.762055	10.507516	681.407213
min	500.000000	25.000000	40.000000	1800.000000
25%	615.000000	27.750000	45.750000	2175.000000
50%	770.000000	29.000000	54.500000	2975.000000
75%	857.500000	32.000000	60.500000	3250.000000
max	960.000000	34.000000	75.000000	3900.000000

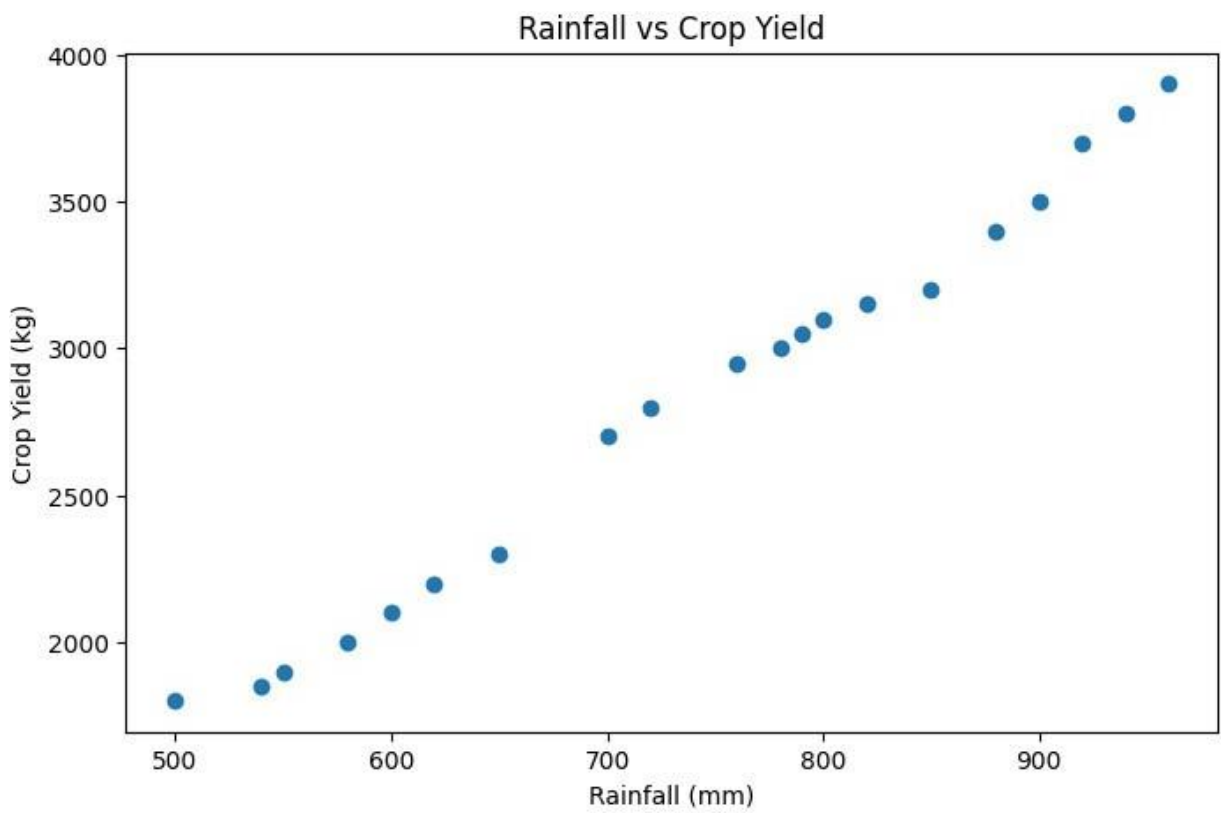
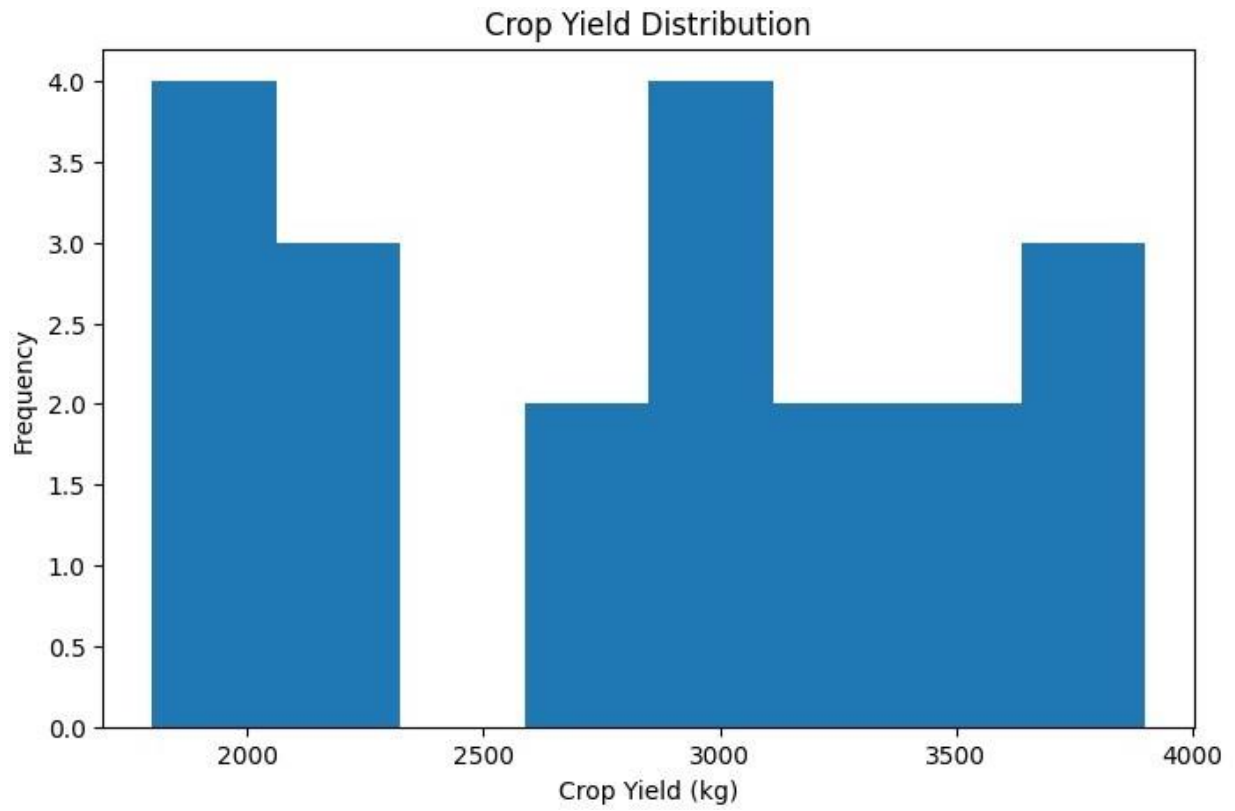
===== CORRELATION =====

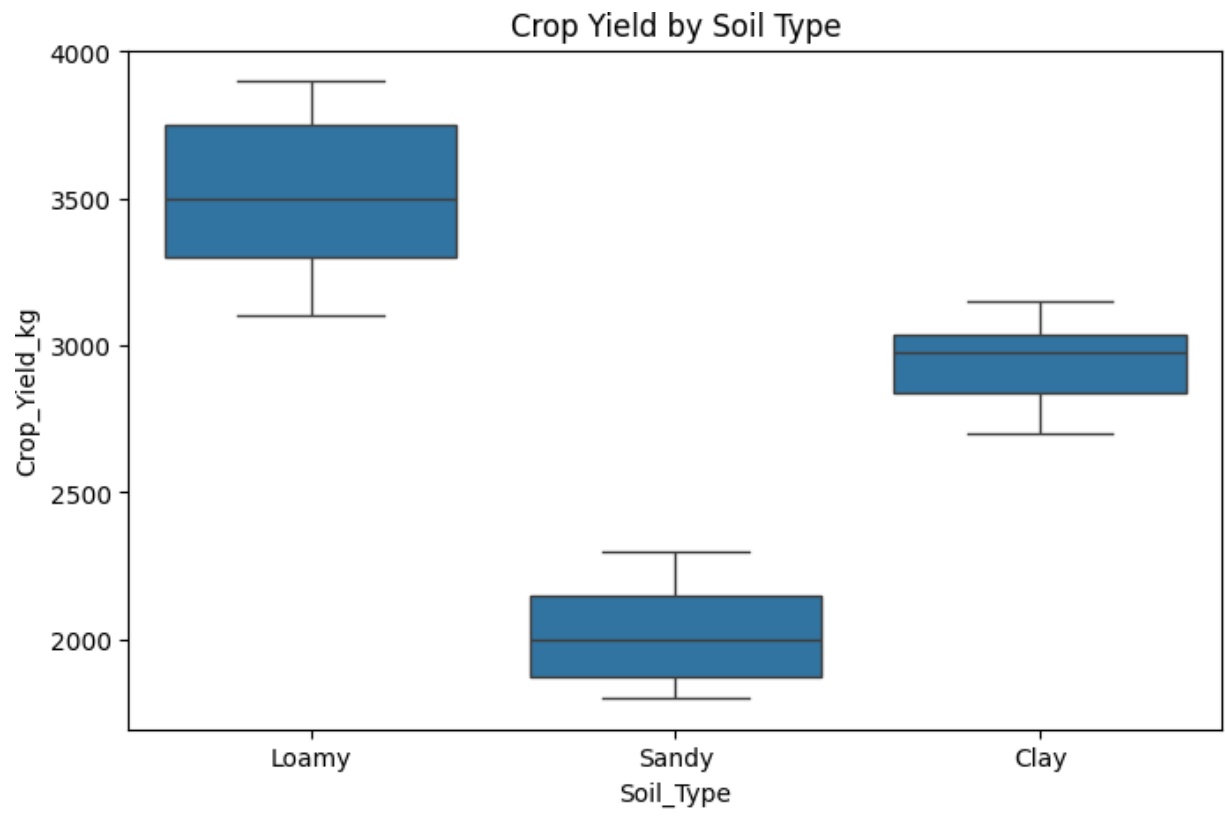
	Rainfall_mm	Temperature	Fertilizer_kg	Crop_Yield_kg
Rainfall_mm	1.000000	-0.993192	0.980550	0.995521
Temperature	-0.993192	1.000000	-0.979735	-0.993300
Fertilizer_kg	0.980550	-0.979735	1.000000	0.983181
Crop_Yield_kg	0.995521	-0.993300	0.983181	1.000000

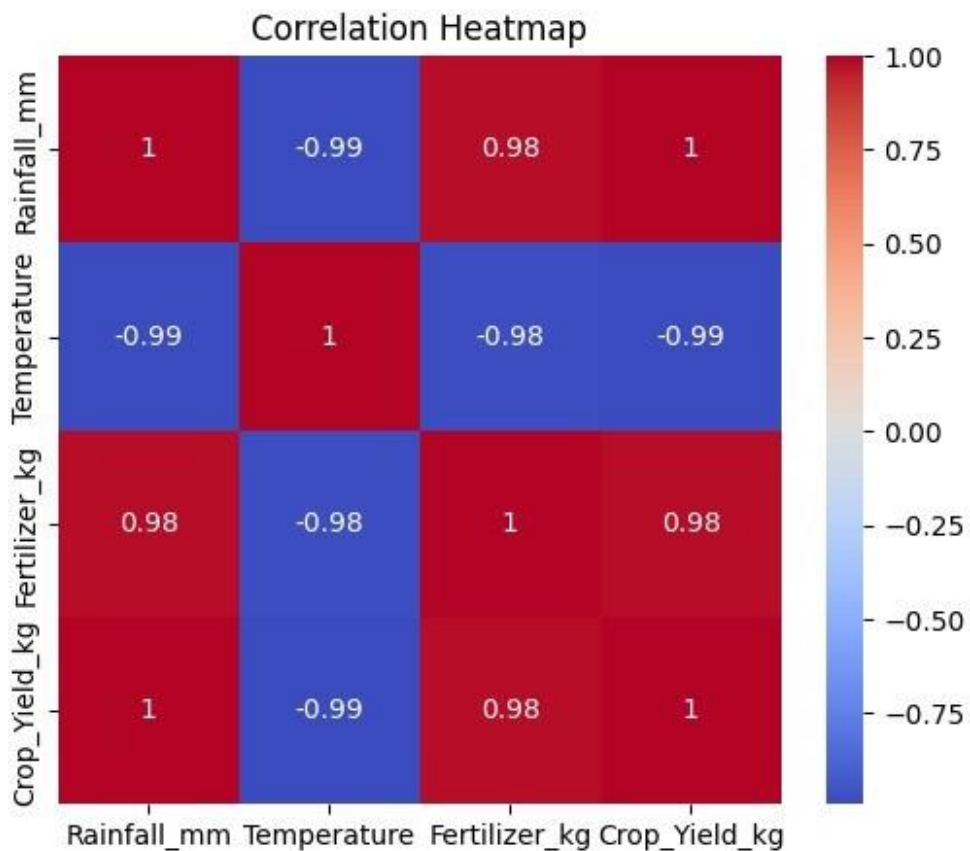


Soil Type Distribution









===== ACTUAL VS PREDICTED =====

	Actual	Predicted
0	3200	3220.95
1	3900	3965.89
2	3150	3268.11
3	2100	2131.01

===== MODEL PERFORMANCE =====

MAE : 58.989247245639376
MSE : 4923.082954847237
RMSE: 70.16468452752594
R2 Score: 0.9880788002606802

